StaffSync – Employee Management System

Full Stack Project Documentation and Phase-wise Report

Prepared by: Atharv Holkar | B.Tech CSE, 3rd Year | **Date:** 27 October 2026

Project Description

StaffSync is a full-stack Employee Management System (HRMS) built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that digitises everyday HR operations of an organisation. It gives administrators and HR personnel a single dashboard to manage employee records, approve or reject leave requests, publish announcements and schedule training programmes, while regular employees can view the company directory, apply for leave, read announcements and enrol in training sessions. The system uses JWT-based authentication (24-hour tokens), bcrypt password hashing, role-based access control (admin, hr, manager, employee), rate-limited login and Helmet-secured HTTP headers, and stores all data in MongoDB Atlas through Mongoose schemas. The React frontend is deployed as a single-page application and talks to a REST API hosted on Render.

Scenario

Consider a growing company with 50 employees whose HR team still tracks leave in WhatsApp groups and spreadsheets. Sana, the HR administrator, logs in to StaffSync, opens the Employees page and adds a new joiner; the system generates a secure temporary password for the new employee. She then posts a holiday notice on the Announcements page and schedules a two-day React workshop on the Training page.

Rahul, a software engineer, logs in with his own credentials. His dashboard shows the latest announcements; he applies for three days of Sick leave with a reason, and enrols himself in the React workshop. Back in her dashboard, Sana sees Rahul's pending request, reviews it and clicks Approve. Rahul instantly sees the status change to Approved in his Leaves page. No paperwork, no lost messages, and every action is restricted by the user's role.

System Requirements

To ensure smooth development, deployment and usage of the StaffSync Web Application, the following prerequisites must be met.

1. Software Requirements

- **Operating System:** Windows 10/11, macOS or Linux – supports cross-platform development and testing.
- **Node.js (v18 or above):** Runtime environment for the Express server and for building the React app. Download from nodejs.org.
- **npm (v8 or above):** Package manager used to install frontend and backend dependencies.
- **React.js (v19):** JavaScript library used to build the dynamic single-page user interface.
- **Express.js (v5):** Lightweight web framework used to build the RESTful API.
- **MongoDB / MongoDB Atlas:** NoSQL database used to store the collections *users, leaves, announcements* and *trainings*. Download from mongodb.com or use a free Atlas cluster.
- **Browser:** Google Chrome / Firefox (latest version) – for rendering and testing the UI.
- **Postman:** Tool for testing the API endpoints during development.
- **Visual Studio Code:** Code editor with built-in Git and terminal support.
- **Git and GitHub:** Version control and collaborative development.

2. Hardware Requirements

Component	Minimum / Recommended	Purpose
Processor	Intel Core i5 (8th Gen+) / AMD Ryzen 5 or better	Fast compilation and multitasking
RAM	8 GB minimum (16 GB recommended)	Running dev servers, IDE and browser together
Storage	At least 1 GB free	Packages, project files and local MongoDB (if used)
Display	1366×768 or higher	Comfortable coding and UI visualisation

Epic 1: Project Architecture (1 hr)

Project architecture defines the structured design of components, their interactions and the technologies used to build and deliver the application.

Technical Architecture

StaffSync follows a classic 3-tier client–server architecture:

Tier	Component	Technology	Responsibility
Presentation	React SPA	React 19, React Router 7, Lucide React icons, CSS	Renders pages (Login, Dashboard, Employees, Attendance, Leaves, Announcements, Training), manages auth state and calls the API
Application	REST API	Node.js, Express 5, Helmet, CORS, express-rate-limit	Routing, input validation, authentication and authorisation, business rules
Security	Auth layer	JWT (jsonwebtoken), bcryptjs	Password hashing, token issue and verification, role checks
Data	Database	MongoDB Atlas with Mongoose 9	Persistent storage of users, leaves, announcements and trainings
Hosting	Cloud	Render (API), static host such as Netlify/Vercel (frontend)	Deployment

Request path: Browser (React UI) → HTTPS request with Bearer token → Express server (Helmet, CORS, JSON parser) → auth middleware → role check (adminOnly where needed) → route handler → Mongoose model → MongoDB Atlas → JSON response → React state update → UI re-render.

ER Diagram

The database has four collections. All relationships point to the **User** collection.

Entity	Key fields	Relationships
User	_id (PK), name, email (unique), password (hash), role, department, position, phone, address, emergency, createdAt, updatedAt	One user applies for many Leaves; authors many Announcements; creates many Trainings; enrols in many Trainings
Leave	_id (PK), applicant (FK → User), type (Casual / Sick / Annual / Unpaid), startDate, endDate, reason, status (Pending / Approved / Rejected)	Many-to-one with User
Announcement	_id (PK), author (FK → User), title, content, timestamps	Many-to-one with User
Training	_id (PK), title, description, trainer, team, mode (Online / Offline / Hybrid), startDate, endDate, enrolled[] (FK → User), createdBy (FK → User)	Many-to-many with User through enrolled; many-to-one with User through createdBy

In-detail explanation: The User collection is the root entity and also the authentication store, so no separate auth table is required. A Leave always belongs to exactly one applicant, which lets the API return only the logged-in employee's own leaves while admins see all. A Training keeps an array of enrolled user ids, so enrol/unenrol is a simple toggle and the participant count is the array length. Mongoose `populate()` joins applicant and enrolled references to readable names at query time.

User Flow

1. User opens the app and is redirected to **/login** if no valid token exists.
2. User enters email and password → API validates, compares the bcrypt hash and returns a JWT plus user details → token is stored in the browser and the user lands on the **Dashboard**.
3. From the sidebar the user can open any module:
   - **Employees:** view directory; admin/HR can search, add and delete employees.
   - **Leaves:** employee applies for leave and tracks status; admin/HR approves or rejects.
   - **Announcements:** everyone reads; admin/HR posts new notices.

- **Training:** everyone views sessions and enrols or unenrols; admin/HR creates and deletes sessions and sees participant lists.
- **Attendance:** attendance view (backend endpoint scaffolded for future extension).

4. User clicks **Logout** → token is removed and the session ends. An expired token (after 24 hours) automatically returns the user to the login page.

Features

- JWT-based secure login with 24-hour session expiry and login rate limiting (10 attempts per 15 minutes).
- Change-password API with current-password verification and an 8-character minimum.
- Dashboard with workforce overview and quick navigation.
- Employee management: list, search, add (auto-generated temporary password) and delete, with protection against deleting your own account.
- Leave management: apply, track and approve/reject with date validation.
- Announcement board with admin/HR publishing.
- Training management: create, enrol/unenrol, participant count and mode (Online/Offline/Hybrid).
- Role-based UI and API protection (admin, hr, manager, employee).
- Production-ready practices: Helmet headers, configurable CORS, request body size limit, hidden error details in production.

Roles and Responsibilities

Role	Responsibilities and permissions
Admin	Full control: manage employees, approve/reject leaves, publish announcements, create/delete trainings, view all leaves and training participants
HR	Same privileges as Admin for day-to-day HR operations
Manager	Logged-in user with directory access, can apply for leave, view announcements and enrol in training

Role	Responsibilities and permissions
Employee	View basic directory (name, email, role, department, position), apply for own leave and view only own leave history, read announcements, enrol in training, change own password

MVC Pattern

The backend follows the Model–View–Controller idea, adapted for a REST API.

Model layer (Mongoose): `models/User.js`, `Leave.js`, `Announcement.js` and `Training.js` define schemas, data types, enums, defaults, required-field validation and references. They are the only layer that talks directly to MongoDB.

Controller layer (business logic): Route handlers receive the request, validate input (email format, date order, role in allowed list, password length, valid ObjectId), call model methods, apply business rules (for example: a user cannot delete themselves, only admin/HR can change leave status) and send a formatted JSON response.

View / Routing layer: Since this is an API there is no HTML view. Express routes (`/api/auth`, `/api/employees`, `/api/leaves`, `/api/announcements`, `/api/trainings`) map HTTP methods to handlers and attach middleware (`auth`, `adminOnly`, `loginLimiter`). The real visual View is the React frontend.

Request flow: Client request → Route → Middleware (auth / role) → Controller logic → Model → MongoDB → Model returns data → Controller formats response → Client receives JSON.

Epic 2: Project Setup and Configuration (30 mins)

Creating the project folder

1. Create a folder named **StaffSync**.
2. Inside it create two folders: **staffsync-frontend** and **staffsync-backend**.
3. Open the StaffSync folder in VS Code.

Client setup (React app)

1. Open the frontend folder in the VS Code terminal: `cd staffsync-frontend`

2. The project is bootstrapped with Create React App (react-scripts 5): `npx create-react-app .`
3. Install the required packages: `npm install react-router-dom lucide-react axios`
4. Start the React dev server: `npm start` (runs on http://localhost:3000).

Server setup (npm init)

1. Open the backend folder: `cd staffsync-backend`
2. Initialise the project: `npm init -y`
3. Install dependencies: `npm install express mongoose cors dotenv bcryptjs jsonwebtoken helmet express-rate-limit`
4. Install the dev dependency: `npm install -D nodemon`
5. Create the file **server.js** and the folders **models** and **routes**.
6. Add scripts in package.json: `"start": "node server.js"` and `"dev": "nodemon server.js"`.

Epic 3: Backend Development (8 hrs)

3.1 Server configuration (server.js)

The server loads environment variables with **dotenv**, refuses to start if `JWT_SECRET` or `MONGO_URI` is missing, enables **Helmet** for secure headers, **CORS** restricted to the origins in `FRONTEND_URL`, and `express.json` with a 10 kb limit. `trust proxy` is enabled so rate limiting sees the real client IP behind Render.

3.2 Third-party libraries and services (explanation)

Library / service	Why it is used
express	Web framework for routing and middleware
mongoose	ODM for MongoDB: schemas, validation, populate
MongoDB Atlas	Cloud-hosted database service
jsonwebtoken	Creates and verifies JWT tokens (24-hour expiry)
bcryptjs	One-way hashing of passwords (10 salt rounds)

Library / service	Why it is used
helmet	Sets secure HTTP response headers
cors	Allows only the frontend origin to call the API
express-rate-limit	Limits login attempts to prevent brute-force attacks
dotenv	Loads secrets from the .env file
nodemon	Restarts the server automatically during development
Render	Cloud platform used to host the API

No paid external APIs are consumed; the only external service is MongoDB Atlas.

3.3 Authentication and authorisation middleware

```
const auth = (req, res, next) => {
  const h = req.headers.authorization || "";
  const t = h.startsWith("Bearer ") ? h.slice(7) : null;
  if (!t) return res.status(401).json({ message: "Not logged in" });
  try { req.user = jwt.verify(t, JWT_SECRET); next(); }
  catch { res.status(401).json({ message: "Session expired, login again" });
}
};
const adminOnly = (req, res, next) =>
  isAdminRole(req.user.role) ? next() : res.status(403).json({ message:
"Admins only" });
```

`auth` extracts and verifies the Bearer token and attaches the decoded user (id, role) to the request. `adminOnly` allows only the admin and hr roles.

3.4 API endpoints

Method	Endpoint	Access	Description
GET	/	Public	Health check
POST	/api/auth/login	Public (rate-limited)	Validates credentials, returns JWT and user

Method	Endpoint	Access	Description
PUT	/api/auth/password	Logged in	Changes own password
GET	/api/profile	Logged in	Returns own profile without password
GET	/api/employees	Logged in	Full list for admin/HR, basic fields for others
POST	/api/employees	Admin/HR	Adds employee, generates temporary password if none given
DELETE	/api/employees/:id	Admin/HR	Deletes employee (not yourself)
GET	/api/leaves	Logged in	All leaves for admin/HR, own leaves for others
POST	/api/leaves	Logged in	Applies for leave
PUT	/api/leaves/:id	Admin/HR	Sets status Pending / Approved / Rejected
GET	/api/announcements	Logged in	Lists announcements, newest first
POST	/api/announcements	Admin/HR	Publishes an announcement
GET	/api/attendance	Logged in	Scaffolded endpoint for the attendance module
GET	/api/trainings	Logged in	Lists trainings with enrolment count and own enrolment status
POST	/api/trainings	Admin/HR	Creates a training session
POST	/api/trainings/:id/enroll	Logged in	Toggles enrolment
DELETE	/api/trainings/:id	Admin/HR	Deletes a training session

3.5 Validation and error handling

Every write endpoint validates input: required fields, email regex, allowed role values, password length, valid MongoDB ObjectId, valid dates and end date not earlier than start date. A central `fail()` helper converts duplicate-key errors into a friendly message and hides internal error details in production. Unknown routes return 404 and malformed JSON returns 400.

3.6 Seeding initial users

`seed.js` connects to MongoDB and creates two demo accounts with hashed passwords: an admin (Sana) and an employee (Rahul). Run it once with `node seed.js`.

Epic 4: Database Development (2 hrs)

1. Create a free cluster on MongoDB Atlas, create a database user and allow network access.
2. Copy the connection string into the backend `.env` file as `MONGO_URI`.
3. Mongoose connects on server start and prints *MongoDB Connected Successfully!*.
4. Collections are created automatically from the models:

Collection	Schema highlights
users	name, unique lowercase email, hashed password, role enum (admin, hr, manager, employee), department, position, phone, address, emergency
leaves	applicant ref, type enum, startDate, endDate, reason, status enum (default Pending)
announcements	author ref, title, content
trainings	title, description, trainer, team, mode enum, startDate, endDate, enrolled[] refs, createdBy ref

All schemas use `timestamps: true`, so createdAt and updatedAt are maintained automatically. The unique index on email prevents duplicate accounts.

Epic 5: Front-End Development (8 hrs)

5.1 Structure

The frontend is a React 19 single-page application using React Router 7. The main file is `src/App.js`, with styles in `App.css` and `index.css`.

5.2 Core utilities

- **API wrapper:** a small `api(path, options)` function adds the Bearer token, parses JSON safely, and throws readable errors (for example when the Render server is waking up).
- **API base URL:** read from `REACT_APP_API_URL`, with the deployed Render URL as fallback.
- **Local storage helper:** stores the token and user under the key *ss_token* with try/catch safety.
- **AuthContext and AuthProvider:** hold the current user, login and logout functions and restore the session on refresh.
- **ProtectedRoute and Layout:** redirect unauthenticated users to /login and wrap pages with the sidebar.

5.3 Pages and components

Route	Component	What it does
/login	Login	Email/password form, shows server errors
/	Dashboard	Summary cards and quick links
/employees	Employees	Directory, search, add and delete (admin/HR)
/attendance	Attendance	Attendance interface
/leaves	Leaves	Apply form, history, approve/reject buttons for admin/HR
/announcements	Announcements	Notice board, post form for admin/HR
/training	Training	Session cards, enrol/unenrol, create/delete for admin/HR

The **Sidebar** shows links with Lucide icons and displays admin-only actions only for admin and hr roles. Libraries used on the client: react-router-dom (routing), lucide-react (icons), axios (listed dependency) and the browser fetch API for calls.

Epic 6: Project Execution (30 mins)

Step 1 – Backend

1. Open a terminal and run `cd staffsync-backend` and `npm install`.
2. Create a `.env` file with:

```
PORT=5000
MONGO_URI=<your MongoDB Atlas connection string>
JWT_SECRET=<long random secret>
FRONTEND_URL=http://localhost:3000
```

3. Optional: `node seed.js` to create demo users.
4. Start the server with `npm run dev` (or `node server.js`). It runs on http://localhost:5000.

Step 2 – Frontend

1. In a new terminal run `cd staffsync-frontend` and `npm install`.
2. Create `.env` with `REACT_APP_API_URL=http://localhost:5000/api`.
3. Run `npm start`. The app opens at http://localhost:3000.

Step 3 – Demo login: sana@staffsync.io / admin123 (admin) and rahul@staffsync.io / emp12345 (employee).

Output screenshots with explanation: *(Insert screenshots of Login, Dashboard, Employees, Leaves, Announcements and Training pages, each with a one-line explanation.)*

Demo video link: *(paste your link here)*

Code drive / GitHub link: *(paste your repository link here)*

Phase-wise Documentation

Field	Details
Project Name	StaffSync – Employee Management System
Team Member	Atharv Holkar (B.Tech CSE, 3rd Year)
Team ID	*(enter your Team ID)*
Date	27 October 2026
Stack	MongoDB, Express.js, React.js, Node.js (MERN)

Phase 1: Ideation Phase

1.1 Define the Problem Statements

The goal of this step is to identify the core problems faced by HR teams and employees in small and mid-sized organisations and use them to guide the development of StaffSync.

Problem Statement	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	an HR administrator	maintain accurate employee records and approve leave quickly	records are scattered across spreadsheets and chats	there is no single, secure system with role-based access	overloaded and worried about errors
PS-2	an employee	apply for leave and know its status	I have to message HR and wait for a reply	there is no transparent request-and-approval workflow	anxious and uncertain about my plans
PS-3	an employee or manager	stay updated about company notices and training sessions	information is shared on informal channels	there is no central announcement and training board	left out and uninformed
PS-4	a company owner	protect sensitive staff data	anyone with a spreadsheet link can see phone numbers and addresses	access is not controlled by role	concerned about privacy and compliance

1.2 Empathy Map Canvas

Understanding the needs, frustrations and behaviours of HR staff and employees who will use StaffSync.

Quadrant	HR Administrator (Sana)	Employee (Rahul)
Says	I need one place to see everyone and approve leave	Did HR see my leave request?

Quadrant	HR Administrator (Sana)	Employee (Rahul)
Thinks	Manual tracking will cause mistakes during audits	I do not know if my leave is approved until the last day
Does	Updates Excel sheets, replies to WhatsApp messages, prints notices	Sends leave requests by message, checks notice boards, asks colleagues about training
Feels	Stressed, overloaded, afraid of data leaks	Uncertain, dependent on others, uninformed
Pains	Duplicate entries, lost requests, no history, no access control	No visibility of request status, missed announcements
Gains	Fast approvals, searchable records, secure role-based access	Instant status updates, one login, easy training enrolment

1.3 Brainstorm and Idea Prioritization

Step 1 – Team gathering, collaboration and problem selection: The problem selected was *digitising core HR operations (employee records, leave, announcements, training) in one secure web application*.

Step 2 – Brainstorm, idea listing and grouping:

Group	Ideas generated
Access and security	JWT login, role-based access, password change, login rate limiting
Employee records	Add, search, delete employees, department and position, emergency contact
Leave	Apply for leave, approval workflow, leave types, status tracking
Communication	Announcement board, email notifications, in-app alerts
Growth	Training sessions, enrolment, certificates
Analytics	Attendance tracking, reports, payroll, charts

Step 3 – Idea prioritization:

Idea	Impact	Effort	Decision
JWT authentication with roles	High	Medium	Build now (Sprint 1)
Employee management	High	Medium	Build now (Sprint 2)
Leave workflow	High	Medium	Build now (Sprint 3)
Announcements	Medium	Low	Build now (Sprint 3)
Training enrolment	Medium	Medium	Build now (Sprint 4)
Attendance tracking	Medium	High	Interface built, full logic in future scope
Email notifications, payroll, reports	Medium	High	Future scope

Phase 2: Project Planning Phase

2.1 Product Backlog and Sprint Schedule

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Member
Sprint-1	Authentication	USN-1	As a user, I can log in with email and password	3	High	Atharv Holkar
Sprint-1	Authentication	USN-2	As a user, my session is secured with JWT and role-based protected routes	3	High	Atharv Holkar
Sprint-1	Dashboard	USN-3	As a user, I can see a dashboard and navigate through a sidebar	2	Medium	Atharv Holkar
Sprint-2	Employee Management	USN-4	As an admin, I can add a new employee	3	High	Atharv Holkar
Sprint-2	Employee Management	USN-5	As a user, I can view and search the employee directory	2	High	Atharv Holkar

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Member
Sprint-2	Employee Management	USN-6	As an admin, I can delete an employee	2	Medium	Atharv Holkar
Sprint-3	Leave Management	USN-7	As an employee, I can apply for leave	3	High	Atharv Holkar
Sprint-3	Leave Management	USN-8	As an admin, I can approve or reject leave	3	High	Atharv Holkar
Sprint-3	Announcements	USN-9	As an admin, I can publish announcements for everyone	2	Medium	Atharv Holkar
Sprint-4	Training	USN-10	As an admin I can create training sessions and as an employee I can enrol	3	Medium	Atharv Holkar
Sprint-4	Account	USN-11	As a user, I can change my password	2	Medium	Atharv Holkar
Sprint-4	Deployment	USN-12	As a team, we deploy the backend on Render and the frontend on a static host	3	High	Atharv Holkar

2.2 Sprint Estimation and Velocity

Sprint	Total Story Points	Duration	Focus
Sprint-1	8	6 days	Authentication, dashboard shell
Sprint-2	7	6 days	Employee management
Sprint-3	8	6 days	Leave and announcements
Sprint-4	8	6 days	Training, password change, deployment
Total	**31**	**24 days**	

Average velocity: 31 story points ÷ 4 sprints = about 7.75 story points per sprint (roughly 1.3 points per day).

Phase 3: Requirement Analysis

3.1 Functional Requirements

FR No.	Functional Requirement (Epic)	Sub-requirement (Story / Sub-task)
FR-1	User Authentication	Login with email and password; JWT session of 24 hours; change password; logout
FR-2	Role-based Access	Roles admin, hr, manager, employee; admin/HR-only actions protected in both UI and API
FR-3	Employee Management	List and search employees; add employee with temporary password; delete employee
FR-4	Leave Management	Apply for Casual, Sick, Annual or Unpaid leave; view history; approve or reject (admin/HR)
FR-5	Announcements	View announcements; publish announcement (admin/HR)
FR-6	Training Management	Create and delete sessions (admin/HR); enrol or unenrol; participant count
FR-7	Attendance	Attendance page and API endpoint prepared for future attendance logic

3.2 Non-functional Requirements

NFR No.	Requirement	Description
NFR-1	Usability	Clean sidebar-based interface with clear icons, forms and error messages
NFR-2	Security	Passwords hashed with bcrypt, JWT authentication, Helmet headers, CORS restriction, login rate limiting, input validation
NFR-3	Reliability	Central error handler, safe JSON parsing, graceful message when the server is waking up

NFR No.	Requirement	Description
NFR-4	Performance	Indexed unique email, lightweight JSON payloads (10 kb limit), populate only needed fields
NFR-5	Scalability	Stateless REST API (JWT) that can run on multiple instances; MongoDB Atlas scales horizontally
NFR-6	Maintainability	Separate Mongoose models, reusable helpers and middleware, environment-based configuration
NFR-7	Portability	Works on any modern browser and any OS; deployable on Render, Netlify or Vercel

3.3 Data Flow Diagram

A Data Flow Diagram shows how data moves between users, processes and data stores in StaffSync.

Level 0 (context diagram): External entities **Admin/HR** and **Employee** send login details, employee data, leave requests, announcements and training enrolments to the **StaffSync System** and receive dashboards, approvals, notices and confirmations.

Level 1 processes and data stores:

Process	Input	Output	Data store
1.0 Authenticate user	Email, password	JWT token and user info	D1 Users
2.0 Manage employees	Employee details (admin/HR)	Directory list, confirmation	D1 Users
3.0 Manage leave	Leave request, status decision	Leave history, updated status	D2 Leaves, D1 Users
4.0 Manage announcements	Title and content	Notice list	D3 Announcements
5.0 Manage training	Session details, enrol request	Session list, enrolment status	D4 Trainings, D1 Users

3.4 User Stories

User Type	Functional Requirement	User Story	Acceptance Criteria	Priority	Release
Admin/HR	Employee Management	As an admin, I can add an employee	Employee appears in the directory and a temporary password is shown once	High	Sprint-2
Employee	Leave Management	As an employee, I can apply for leave	Request is saved with status Pending	High	Sprint-3
Admin/HR	Leave Management	As an admin, I can approve or reject a request	Status updates and the employee sees it	High	Sprint-3
Employee	Announcements	As an employee, I can read announcements	Latest notices appear first	Medium	Sprint-3
Employee	Training	As an employee, I can enrol in a training	Enrolled state and count update instantly	Medium	Sprint-4
User	Account	As a user, I can change my password	Password changes only when the current password is correct	Medium	Sprint-4

3.5 Technology Stack

Table 1: Components and technologies

S.No	Component	Description	Technology
1	User Interface	Web interface for admin, HR and employees	React 19, React Router 7, CSS, Lucide React
2	Application Logic	REST API, validation, authorisation	Node.js, Express 5
3	Security	Authentication and password protection	JWT, bcryptjs, Helmet, express-rate-limit
4	Database	Stores users, leaves, announcements, trainings	MongoDB Atlas, Mongoose 9
5	Hosting	Deployment of API and frontend	Render, Netlify/Vercel

Table 2: Application characteristics

S.No	Characteristic	Description	Technology
1	Open-source frameworks	Frontend and backend frameworks	React.js, Express.js
2	Architecture	3-tier client–server with REST API	MVC-style layers
3	Authentication	Stateless token-based	JWT
4	Version control	Source management	Git and GitHub

Phase 4: Project Design Phase

4.1 Problem–Solution Fit

Purpose: Ensure StaffSync addresses the real gaps in HR operations of small organisations.

Problem	StaffSync solution
Scattered employee records	Central employee directory with search and secure storage
Slow, untraceable leave approval	Digital leave workflow with Pending, Approved and Rejected statuses
Missed announcements	Central announcement board
Unplanned training participation	Training sessions with online enrolment
Unauthorised access to personal data	JWT login and role-based API protection

4.2 Proposed Solution

S.No	Parameter	Description
1	Problem Statement	HR teams rely on spreadsheets and chats for employee data, leave and communication, causing delays, errors and privacy risks.

S.No	Parameter	Description
2	Idea / Solution Description	StaffSync is a MERN-stack web application with role-based access that manages employees, leave, announcements and training in one place.
3	Novelty / Uniqueness	Auto-generated temporary passwords, role-filtered data (employees see only basic directory info), one-click training enrol toggle, hardened API (Helmet, rate limiting).
4	Social Impact / Customer Satisfaction	Saves HR time, gives employees transparency, protects personal data and promotes a paperless workplace.
5	Business Model	Free tier for small teams; subscription per employee for advanced modules such as payroll, reports and email alerts.
6	Scalability	Stateless API and MongoDB Atlas allow growth; modular models make it easy to add attendance, payroll and notifications.

4.3 Solution Architecture

StaffSync uses a scalable 3-tier architecture: the React single-page application (presentation), the Express REST API with authentication and role middleware (business logic) and MongoDB Atlas (data). Key goals:

- Secure login and fine-grained authorisation for every request.
- Clear separation of frontend, API and database so each can be deployed and scaled independently.
- Stateless design using JWT so the API can run on multiple instances.
- Simple REST contract that future mobile apps can reuse.

Phase 5: Performance Testing and User Acceptance Testing (UAT)

Project: StaffSync – Employee Management System | **Version:** v1.0.0

Testing scope: login and logout, role-based access, employee management, leave workflow, announcements, training enrolment, change password and input validation.

Testing environment: Local setup (frontend http://localhost:3000, backend http://localhost:5000) or the deployed Render and static-host URLs. Test accounts: Admin sana@staffsync.io / admin123 and Employee rahul@staffsync.io / emp12345.

Test Cases

ID	Scenario	Test Steps	Expected Result	Actual Result / Status
TC-01	Valid login	Open /login, enter correct credentials, submit	User is redirected to the Dashboard	*(Pass / Fail)*
TC-02	Invalid login	Enter a wrong password	Message *Invalid credentials* is shown	*(Pass / Fail)*
TC-03	Login rate limit	Submit wrong credentials more than 10 times in 15 minutes	Message *Too many login attempts* is shown	*(Pass / Fail)*
TC-04	Protected route	Open /employees without logging in	Redirected to /login	*(Pass / Fail)*
TC-05	Add employee (admin)	Employees > Add, fill name, valid email, role, submit	Employee is created and a temporary password is shown	*(Pass / Fail)*
TC-06	Duplicate email	Add an employee with an existing email	Message *Email already exists*	*(Pass / Fail)*
TC-07	Role restriction	Log in as employee and open Employees	No add or delete options; API returns 403 for POST	*(Pass / Fail)*
TC-08	Delete employee	Admin deletes another employee	Employee is removed; deleting own account is blocked	*(Pass / Fail)*
TC-09	Apply leave	Employee submits type, dates and reason	Request saved with status Pending	*(Pass / Fail)*

ID	Scenario	Test Steps	Expected Result	Actual Result / Status
TC-10	Invalid dates	Enter end date earlier than start date	Message *End date cannot be before start date*	*(Pass / Fail)*
TC-11	Approve leave	Admin clicks Approve on a pending request	Status becomes Approved and the employee sees it	*(Pass / Fail)*
TC-12	Leave privacy	Employee opens Leaves	Only own requests are visible	*(Pass / Fail)*
TC-13	Post announcement	Admin posts title and content	Notice appears at the top for everyone	*(Pass / Fail)*
TC-14	Create training	Admin creates a session with valid dates	Session appears in the Training list	*(Pass / Fail)*
TC-15	Enrol toggle	Employee clicks Enrol, then again	Enrolment count increases, then decreases	*(Pass / Fail)*
TC-16	Weak new password	Change password with fewer than 8 characters	Message *New password must be at least 8 characters*	*(Pass / Fail)*
TC-17	Logout	Click Logout	Token removed, redirected to /login	*(Pass / Fail)*

Performance Observations

Parameter	Observation to record
API response time (login, list employees)	Measure in Postman or browser dev tools
First request on Render free tier	May take around 30 seconds when the instance is asleep
Frontend load	Check Lighthouse score in Chrome

Sign-off

Once all test cases are executed, record the results above, then mark the project as **accepted** if there are no open high-priority defects.

Conclusion

StaffSync successfully demonstrates a secure, role-based HR management platform covering authentication, employee management, leave workflow, announcements and training. It applies MERN-stack concepts, REST API design, JWT security and cloud deployment in a real-world scenario.

Future Enhancements

- Complete the attendance module with check-in/check-out and monthly reports.
- Split route handlers into dedicated controller files for a stricter MVC structure.
- Email and in-app notifications for leave decisions and new announcements.
- Payroll, performance reviews and analytics dashboards.
- Automated testing (Jest and Supertest) and CI/CD pipeline.

References

- React documentation (react.dev) and React Router documentation
- Express.js and Mongoose documentation
- MongoDB Atlas documentation
- JSON Web Token (jwt.io) and OWASP authentication guidelines
- Render deployment documentation